



CIN0136

Desenvolvimento de Software

Desenvolvimento de Software

Prof. Vinicius Cardoso Garcia

vcg@cin.ufpe.br :: @vinicius3w :: assertlab.com

<http://viniciusgarcia.me>



**Centro de
Informática**
UFPE



UNIVERSIDADE
FEDERAL
DE PERNAMBUCO

Licença do material

Este Trabalho foi licenciado com uma Licença



**Atribuição-NãoComercial-
Compartilha Igual 4.0 Internacional
(CC BY-NC-SA 4.0)**

Mais informações visite

<https://creativecommons.org/licenses/by-nc-sa/4.0/deed.pt> BR



Engenharia de Software em Dimensões

É um material abrangente, didático e atualizado que apresenta a Engenharia de Software como uma disciplina multidimensional. Cada capítulo foca em uma dimensão prática, conectando teoria e realidade profissional com clareza, profundidade e propósito.

Prof. Vinicius Cardoso Garcia

<https://bit.ly/esdbook>

Engenharia de Software em Dimensões

Da base conceitual à prática profissional



Ponto de Commit

O podcast figital da disciplina (IF977) Engenharia de Software do curso de bacharelado em Sistemas de Informação do Centro de Informática (CIn) da Universidade Federal de Pernambuco (UFPE).

<https://go.uaite.xyz/ponto-de-commit>

Ponto de Commit



O podcast figital da disciplina de Engenharia de Software



Engenharia de Software Moderna

Engenharia de Software Moderna é um livro destinado a alunos de cursos de graduação em Computação. Ele pode ser lido também por profissionais que buscam um conhecimento horizontal da área.

Prof. Marco Tulio Valente

<https://engsoftmoderna.info>

@engsoftmoderna

ENGENHARIA — DE — SOFTWARE MODERNA

Princípios e práticas para
desenvolvimento de software
com produtividade



MARCO TULIO VALENTE



Medição de Qualidade e Cultura de Qualidade Contínua

Como demonstrar, medir e sustentar qualidade ao longo do ciclo de desenvolvimento.



Visão Geral da Aula

De segunda para hoje

Na segunda-feira, o foco foi "o que é qualidade". Hoje, o foco muda para "como medir e garantir qualidade". A progressão é intencional: primeiro entendemos o conceito, depois aprendemos a operacionalizá-lo.

Por que agora?

A partir de agora, cada equipe já pode:

- Definir os atributos de qualidade prioritários do projeto
- Escrever o Definition of Done do Sprint 1
- Iniciar o Sprint 1

Esta aula fornece a base conceitual para essas decisões práticas.



Objetivos de Aprendizagem

Ao final desta aula, você será capaz de:

1

Descrever técnicas de medição

Conhecer pelo menos 3 técnicas de medição de qualidade e saber quando aplicar cada uma no contexto do projeto.

2

Defender qualidade contínua

Articular por que qualidade contínua é mais eficaz, e mais barata, do que a verificação feita apenas no final.

3

Analisar critérios Gherkin

Revisar criticamente os cenários escritos na Semana 4 e identificar quais atributos de qualidade estão faltando.

4

Preparar o Definition of Done

Chegar à sessão prática de quinta-feira com uma proposta concreta de DoD para discutir com a equipe.



Retomada: A Pergunta que Ficou

"Quando entregarmos o produto ao stakeholder, como ele saberá se tem qualidade?"

Essa era a lição de casa. Respostas típicas giram em torno de "*funcionar direitinho*", "*ser bonito*", "*não travar*". São percepções **válidas**, mas **insuficientes** para um profissional de engenharia.



Sentir vs. Demonstrar Qualidade



O stakeholder **sente** qualidade

Percepção subjetiva, baseada na experiência de uso. Importante, mas não controlável diretamente pela equipe de desenvolvimento.

O profissional **demonstra** qualidade

Com **evidências concretas**: critérios de aceitação atendidos, código revisado por pares, bugs identificados e resolvidos, feedback registrado. Qualidade se demonstra com dados, não com afirmações.

❏ A diferença entre "achamos que está bom" e "podemos provar que está bom" é o que separa amadores de profissionais.



Métricas de Qualidade de Software

São **medidas quantitativas** usadas para **avaliar a qualidade** do software.

Fornecem uma maneira de **avaliar objetivamente** diferentes aspectos da qualidade do software, incluindo confiabilidade, eficiência, segurança, manutenibilidade e tamanho.

Permitem **quantificar** aspectos da qualidade do software que podem ser **subjetivamente**. Elas também nos permitem **rastrear a qualidade do software ao longo do tempo** e identificar áreas que podem precisar de melhoria.



O que é Medição de Qualidade?

A medição de qualidade de software é uma prática para garantir que os sistemas desenvolvidos atendam aos padrões esperados.

Elas ajudam a identificar lacunas e alinhar as soluções às expectativas dos stakeholders, fornecendo dados objetivos para tomada de decisão e melhoria contínua dos processos de desenvolvimento.

Definir Indicadores Chave

Estabelecer métricas relevantes para os critérios de qualidade prioritários do projeto.

Implementar Ferramentas de Medição

Selecionar e configurar ferramentas adequadas para coletar dados de qualidade automaticamente.

Analisar Resultados

Interpretar os dados coletados para identificar tendências, problemas e oportunidades de melhoria.

Aplicar Melhorias

Implementar mudanças baseadas nas análises para elevar continuamente a qualidade do software.



99.9%

Percentual de tempo em que o sistema está operacional, medido em uptime.

85%

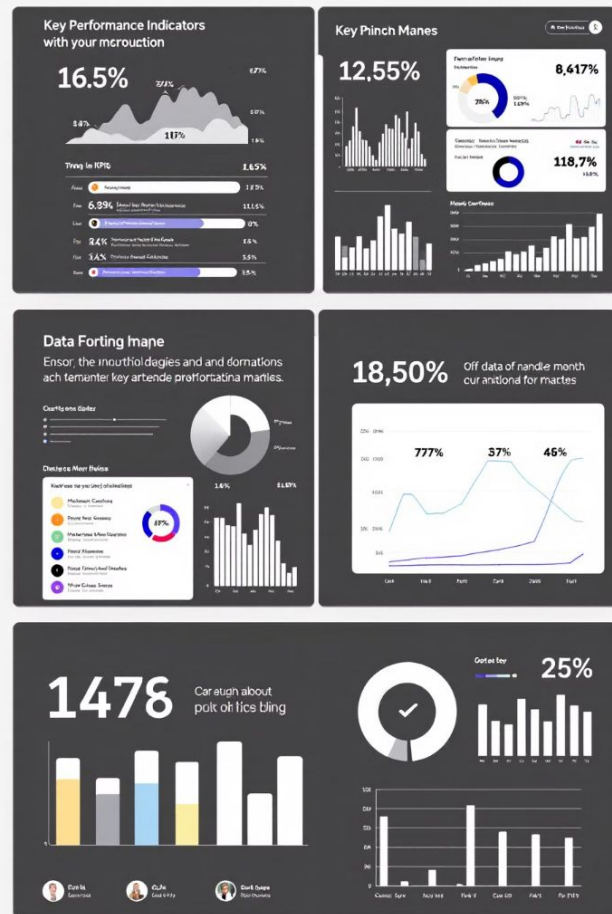
Índice baseado em pesquisas
qualitativas com usuários finais.

1.2s

Velocidade média de
processamento de requisições do
usuário.

0.5%

Frequência de erros ou interrupções no sistema em produção.



Técnicas de Medição de Qualidade

Um repertório de ferramentas e práticas que os profissionais usam para tornar qualidade mensurável.

Para cada técnica, pense: **isso é algo que já faço, vou fazer em breve, ou não está no horizonte?**



Métricas de Código

Complexidade ciclomática, linhas por função, duplicação.

Ferramentas como SonarQube medem automaticamente.

Quando o ESLint (Semana 6) reclamar de uma função muito longa, é uma **métrica de qualidade falando**.



Análise Estática

ESLint e Prettier encontram problemas **sem executar o código**. Entram no projeto na Semana 6, mas o conceito é importante já: prevenção antes da execução.



Revisão por Pares

Code review: antes de integrar código novo, outro membro da equipe lê, questiona e aprova.

Não é carimbo, é conversa sobre decisões. Vocês vão praticar ao longo dos sprints.



Técnicas de Medição — Continuação



Critérios de Aceitação (Gherkin)

Vocês já têm critérios escritos desde a Semana 4. Cada

Dado/Quando/Então é uma declaração de qualidade.

Na Semana 9, esses cenários viram testes automatizados com Playwright. Mas já agora funcionam como referência verificável.



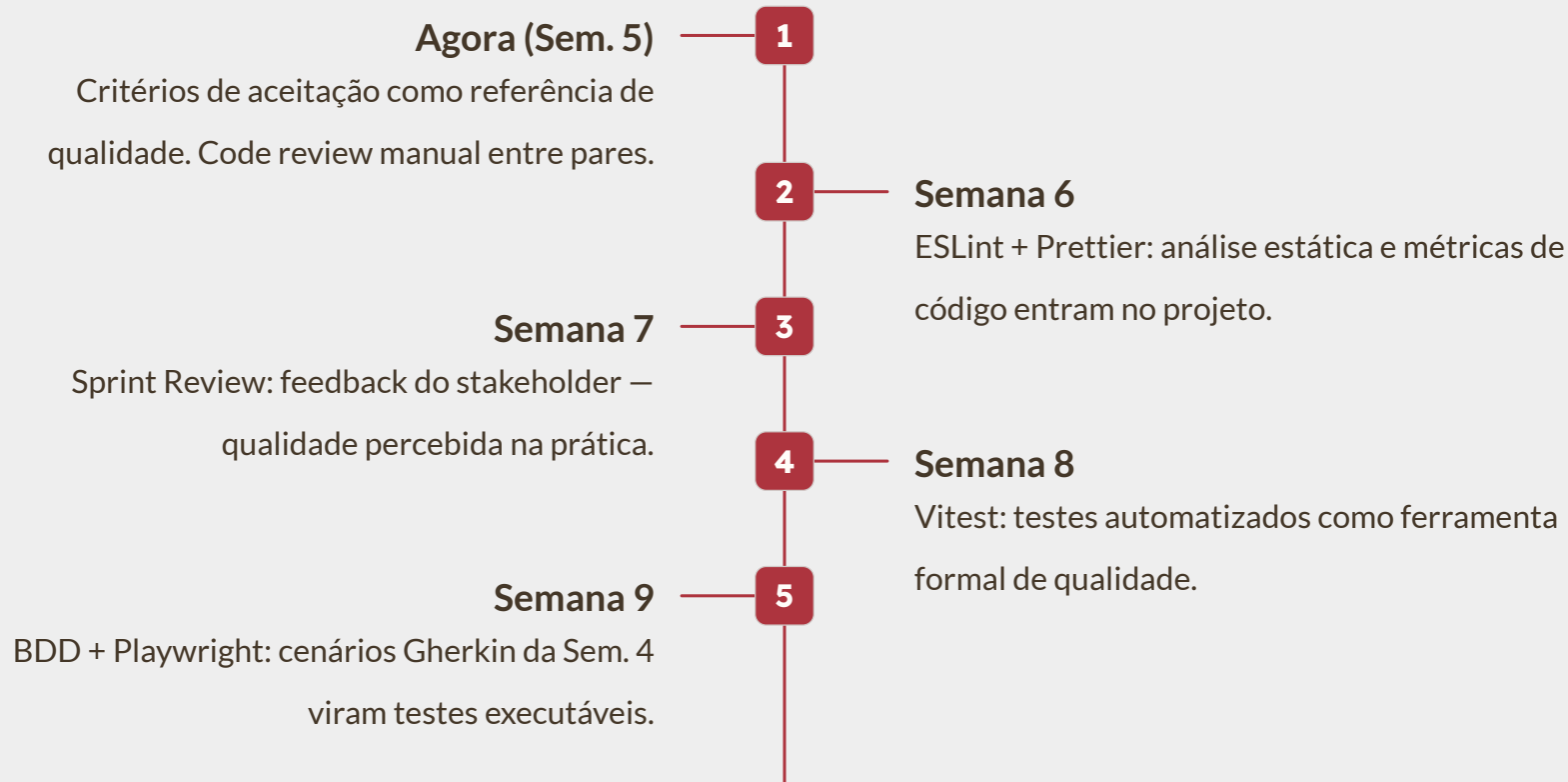
Feedback do Usuário

A Sprint Review com o stakeholder é o **teste de qualidade mais honesto**. O stakeholder não se importa com métricas internas, ele se importa com o que vê e usa. Primeira Sprint Review: Semana 7.

- ❏ Estas técnicas formam um ecossistema complementar, nenhuma sozinha é suficiente. Profissionais combinam várias ao longo do ciclo de desenvolvimento.



Quando Cada Técnica Entra no Semestre



Análise dos Critérios Gherkin da Semana 4

Este é o bloco mais importante da aula. Abram os critérios de aceitação que escreveram no Strateegia na Semana 4.

- 📄 **Tarefa:** Escolham 3 critérios de aceitação. Para cada um: qual atributo de qualidade ele captura? E o que ele **NÃO** captura? Trabalho em equipe por 10 minutos, depois compartilhamos no quadro.



O Padrão Esperado: Funcionalidade em Primeiro Lugar

O que vocês provavelmente escreveram

A maioria dos critérios captura **funcionalidade**: o sistema faz X quando Y acontece. Exemplo típico:

"Dado que o usuário está logado, quando ele clica em salvar, então o registro é criado."

✓ Funcionalidade coberta. Isso é ótimo.

O que provavelmente ficou de fora

Desempenho: E se demorar 30 segundos para salvar?

Confiabilidade: E se o registro for criado mas não aparecer na lista?

Segurança: E se qualquer pessoa puder acessar o registro de outra?

Usabilidade: O usuário sabe que o salvamento foi concluído?



Os Critérios Não Estão Errados — Estão Incompletos

Na Semana 4

O foco era **funcionalidade**. Faz sentido, era o primeiro contato com Gherkin. Os critérios cumpriram o objetivo daquele momento.

Agora (Semana 5)

Com os **atributos de qualidade** em mente, vocês podem revisar e expandir os critérios para cobrir desempenho, segurança, confiabilidade e usabilidade.

A seguir...

Ao definir o **Definition of Done**, incluam critérios que vão além do "funciona". Essa é a evolução natural da prática.

- ❑ Reconhecer lacunas nos próprios artefatos é uma habilidade profissional central. Isso não é crítica — é maturidade técnica.



Mapeando os Atributos de Qualidade nos Critérios

Use esta referência para analisar seus critérios Gherkin e identificar quais dimensões estão cobertas e quais precisam de atenção:

Atributo	Pergunta a fazer ao critério	Exemplo de lacuna típica
Funcionalidade	O sistema faz o que deve fazer?	Geralmente coberta
Desempenho	Em quanto tempo? Com que carga?	Raramente mencionada
Confiabilidade	E se algo der errado?	Casos de erro ignorados
Segurança	Quem pode acessar o quê?	Quase nunca abordada
Usabilidade	O usuário entende o que aconteceu?	Feedback visual ausente
Manutenibilidade	Outro dev consegue estender isso?	Fora do escopo do Gherkin



Qualidade Contínua vs. Verificação Final

Por que corrigir problemas tarde custa mais em tempo, esforço e confiança da equipe.



Os Dois Cenários

● Cenário A — Sem verificação contínua

A equipe desenvolve sem parar para verificar qualidade. No final do sprint, descobre problemas acumulados: critérios não atendidos, código difícil de integrar, regressões não detectadas.

Resultado: 5 dias de desenvolvimento + 2 dias de correção apressada = entrega atrasada e estressante.

● Cenário B — Com verificação contínua

A equipe verifica qualidade em cada feature: code review, critérios verificados, linter sem warnings. Problemas são encontrados em minutos, não em dias.

Resultado: 3 dias de desenvolvimento + 10 min por feature = entrega mais rápida e mais confiável.

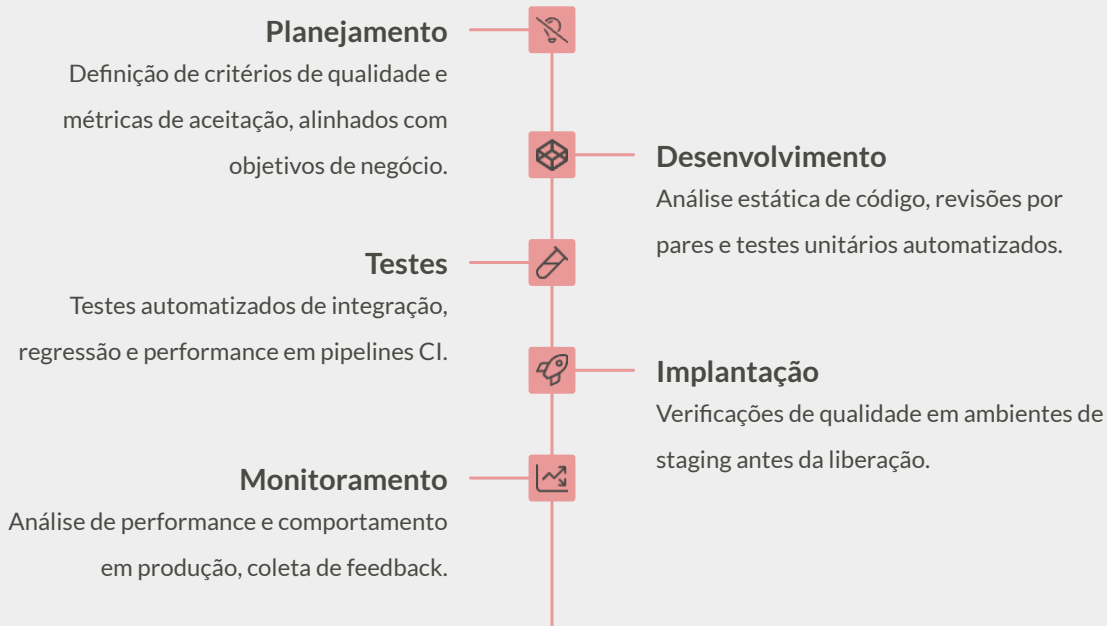
- ❏ O custo de corrigir um bug cresce exponencialmente quanto mais tarde ele é encontrado. Encontrar em code review custa minutos. Encontrar na Sprint Review custa horas. Encontrar em produção pode custar dias.



Integração da Qualidade no Ciclo de Vida

A integração da avaliação contínua no ciclo de vida do software requer a implementação de práticas de qualidade em cada estágio, desde o planejamento inicial até o monitoramento pós-implantação, criando um fluxo contínuo de feedback e melhorias.

A avaliação contínua reduz custos ao prevenir problemas antes que impactem os usuários finais e facilita a adaptação às mudanças no escopo do projeto, permitindo ajustes rápidos baseados em feedback real.



O Definition of Done como Ferramenta de Qualidade Contínua

O **Definition of Done (DoD)** é a forma mais prática de operacionalizar qualidade contínua. É uma lista de critérios que *toda feature* precisa atender para ser considerada "pronta" — sem exceções.

→ **Qualidade deixa de ser julgamento**

Com um DoD claro, ninguém precisa decidir "isso está bom o suficiente?", a resposta está nos critérios. Isso reduz conflitos e aumenta consistência.

→ **Qualidade é verificada antes da integração**

O DoD impede que código incompleto entre na branch principal. Cada item da lista é um portão de qualidade.

→ **Qualidade evolui com a equipe**

O DoD não é fixo. Na Semana 8, quando testes entrarem, ele vai evoluir naturalmente. A equipe aprende e o DoD reflete esse aprendizado.



Exemplos de Itens para o DoD do Sprint 1

Estes são exemplos — não uma lista prescritiva. **Cada equipe decide o que faz sentido para seu contexto.**

O valor pedagógico está na decisão coletiva.

Processo

- Código revisado por pelo menos 1 colega
- Pull request com descrição clara

Funcionalidade

- Critérios de aceitação verificados manualmente
- Feature testada no ambiente de desenvolvimento

Código

- Sem warnings do linter (quando configurado)
- Sem console.log esquecido

Documentação

- README atualizado se necessário
- Comentários em lógica não óbvia



Como Definir um DoD Eficaz



O DoD mais eficaz começa pequeno e realista, e cresce com a maturidade da equipe. Um DoD ambicioso demais que ninguém cumpre é pior do que nenhum DoD.



Relação entre Qualidade e Processos de Desenvolvimento

A qualidade do software é diretamente influenciada pela maturidade e eficácia dos processos de desenvolvimento. Processos bem definidos estabelecem padrões claros, promovem consistência e facilitam a identificação precoce de problemas. Eles também suportam práticas de engenharia sólidas, como revisões de código e testes automatizados.

Uma cultura organizacional que valoriza a qualidade e o compromisso da equipe em seguir boas práticas são a base para processos eficazes e, consequentemente, para software de alta qualidade.



Mais do que uma Prática Técnica

A avaliação contínua de qualidade transcende o aspecto puramente técnico, representando um elemento estratégico que conecta equipes e processos em busca de excelência. Ela se torna parte da cultura organizacional, influenciando decisões em todos os níveis e promovendo uma mentalidade de melhoria constante.

Alinhamento Estratégico

Conecta práticas técnicas com objetivos de negócio, garantindo que a qualidade contribua diretamente para o valor entregue aos clientes e para a vantagem competitiva da organização.

Transformação Cultural

Promove uma cultura onde todos são responsáveis pela qualidade, não somente equipes de teste, criando um ambiente de colaboração e aprendizado contínuo.

Melhoria Contínua

Estabelece ciclos de feedback que permitem aprimoramento constante, tanto do produto quanto dos processos de desenvolvimento, adaptando-se às mudanças do mercado.





Questão Estruturante

"Se qualidade é tão importante, por que tantas empresas entregam software com bugs? O que impede a qualidade na prática: falta de competência, falta de tempo, ou falta de cultura?"

Debate aberto por 3 minutos. Não existe resposta certa, a tensão é intencional e vai reaparecer ao longo de todo o semestre.



Conexão com as Próximas Aulas

09/04 – Quinta-feira

Lab prático: atributos de qualidade prioritários +

Definition of Done + início Sprint 1

Semana 7

Sprint 1 Review com stakeholder: primeira

demonstração real de qualidade percebida.

Semana 9

BDD + Playwright: os cenários Gherkin da Semana 4

viram testes executáveis.

Semana 6

Design e Arquitetura: organização de código operacionaliza manutenibilidade. ESLint e Prettier entram no projeto.

Semana 8

Lab Vitest: testes automatizados como ferramenta formal de qualidade.



Preparação para a sequência



A aula prática de quinta é o momento de transformar conceitos em decisões reais de equipe. Cheguem preparados:

1 Releiam os critérios Gherkin

Abram o Strateegia e revisitem os cenários escritos na Semana 4. Identifiquem quais atributos de qualidade estão cobertos e quais estão faltando.

2 Pensem no que o stakeholder mais valoriza

Considerando o produto do projeto, qual atributo de qualidade seria mais crítico para o stakeholder: desempenho, segurança, usabilidade, confiabilidade?

3 Tragam uma proposta de DoD

Cada membro da equipe deve chegar com pelo menos 3 sugestões de itens para o Definition of Done. A equipe debate e decide juntos.



Resumo: Os Três Pilares desta Aula

Medir qualidade

Qualidade não é sentimento, é evidência. Métricas de código, análise estática, revisão por pares, critérios de aceitação e feedback do usuário são as ferramentas do profissional.

Analisar criticamente

Os critérios Gherkin da Semana 4 capturam funcionalidade, mas qualidade é mais ampla. Desempenho, segurança, confiabilidade e usabilidade também precisam de atenção.

Agir continuamente

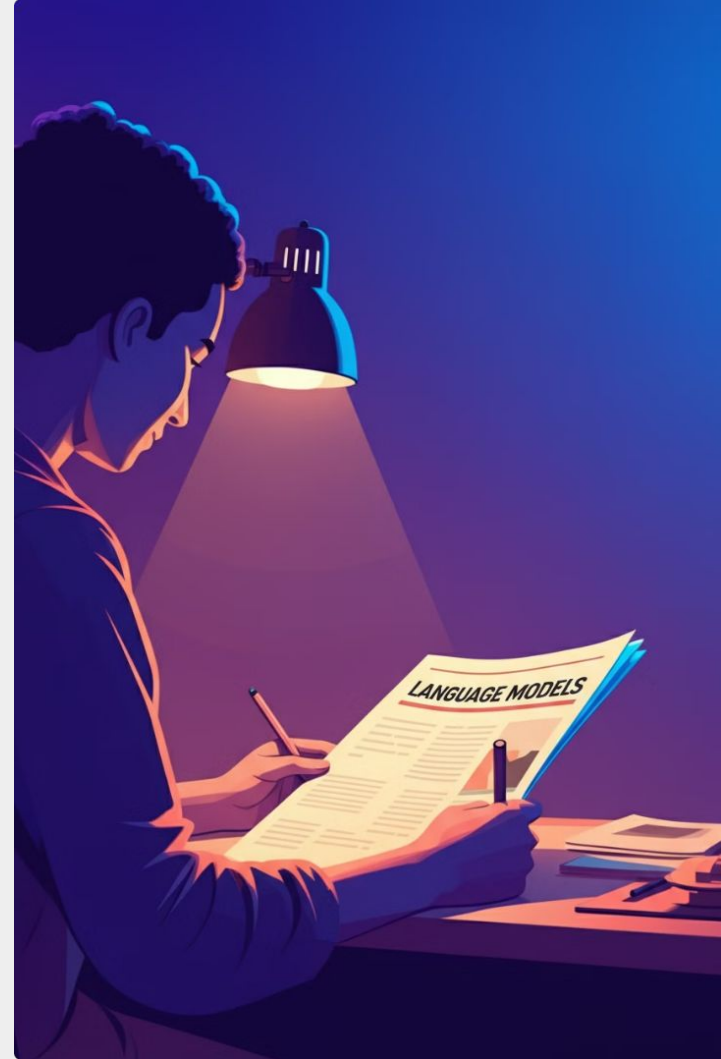
Qualidade contínua é mais barata e eficaz que verificação tardia. O Definition of Done é a ferramenta prática que transforma essa convicção em processo.

📅 **Lembre:** Quinta-feira Santa = sem aula. A prática acontece em **09/04**. Venham preparados para decidir, não apenas para ouvir.



Próximos passos

- Eng. Software em Dimensões
 - Cap. 8 (seções 8.1–8.3):
Qualidade de software,
critérios, medição
- Eng. Software Moderna
 - Cap. 8 — Testes: por que
qualidade importa; relação
entre testes e qualidade



Vamos debater?

Qualidade de Software

introdução sobre o tema

questões

referências

análises

 **ativar modo anônimo**

o modo anônimo permite responder as questões temporariamente de forma anônima. ao desabilitar, as pessoas que responderam passam a ser identificadas.

questão 1

Seu stakeholder diz que o software tem qualidade. Você, como desenvolvedor, discorda. Quem decide?

0 respostas

questão 2

É possível medir qualidade de software de forma objetiva? Ou toda avaliação de qualidade é, no fundo, subjetiva?

0 respostas

 adicionar questão

No contexto do desenvolvimento de sistemas, a qualidade de software emerge como um pilar fundamental para garantir o êxito de produtos e serviços que, cada vez mais, permeiam a vida cotidiana e os processos organizacionais. O conceito de qualidade de software, contudo, transcende a mera operacionalidade de um sistema. A noção de considerar que um software é de qualidade simplesmente porque "funciona" revela-se inadequada frente ao grau de complexidade e ao conjunto diversificado de expectativas que permeiam o desenvolvimento de soluções tecnológicas atuais. Embora a funcionalidade seja um aspecto crítico, a qualidade de software abrange um espectro mais amplo de critérios, sendo imprescindível compreender suas múltiplas dimensões para assegurar o atendimento completo aos requisitos de negócio e à satisfação dos stakeholders.

Para abordar qualidade de software de forma abrangente, é necessário considerar um conjunto de critérios estabelecidos ao longo de décadas de pesquisa e prática na engenharia de software. Elementos como funcionalidade, desempenho, usabilidade, confiabilidade, segurança e manutenibilidade compõem esse arcabouço. Funcionalidade refere-se à capacidade do software de atender corretamente às demandas especificadas; desempenho envolve aspectos como

Medição de qualidade e cultura de qualidade...

introdução sobre o tema

questões

referências

análises

 **ativar modo anônimo**

o modo anônimo permite responder as questões temporariamente de forma anônima. ao desabilitar, as pessoas que responderam passam a ser identificadas.

introdução sobre o tema

A busca pela excelência no desenvolvimento de software demanda não apenas a implantação de práticas robustas de engenharia, mas também a implementação de mecanismos eficazes de medição e cultura de qualidade contínua. O conceito de qualidade, embora multidimensional e muitas vezes relativo, pode e deve ser sistematicamente mensurado e acompanhado ao longo do ciclo de vida do produto. A medição contínua permite que equipes não apenas identifiquem desvios, mas também promovam melhorias incrementais e sustentáveis, fomentando tanto eficiência operacional quanto a satisfação dos stakeholders. Nesse contexto, torna-se imprescindível adotar técnicas e métricas simples, porém poderosas, capazes de traduzir o estado da arte da qualidade nos processos e nos produtos entregues.

Entre as métricas amplamente difundidas para avaliação da qualidade de software destacam-se a complexidade, a duplicação e a cobertura. Métricas de complexidade, como o Índice de McCabe, indicam os pontos do código mais propensos a erros e de difícil manutenção, servindo como sinalizadores essenciais para refatorações. Já métricas de duplicação sinalizam trechos de código repetido, combatendo a proliferação de inconsistências. Por fim, a cobertura –

questão 1

Se qualidade é tão importante, por que tantas empresas entregam software com bugs? O que impede a qualidade na prática: falta de competência, falta de tempo, ou falta de cultura?

2 respostas · 3 comentários

questão 2

Vamos debater?



contacts

vcg@cin.ufpe.br
@vinicius3w
viniciusgarcia.me

academy

go.uaite.xyz/vcg-es
go.uaite.xyz/vcg-microservices
go.uaite.xyz/vcg-devops
go.uaite.xyz/vcg-esaia
go.uaite.xyz/vcg-deso

Desenvolvimento de Software (CIN0136)

Medição de Qualidade e Cultura
de Qualidade Contínua